

Fundamentos de Aprendizagem Automática

Class 01 — Learning as Compression, Pitfalls & Taxonomies

Mário Antunes

Universidade de Aveiro

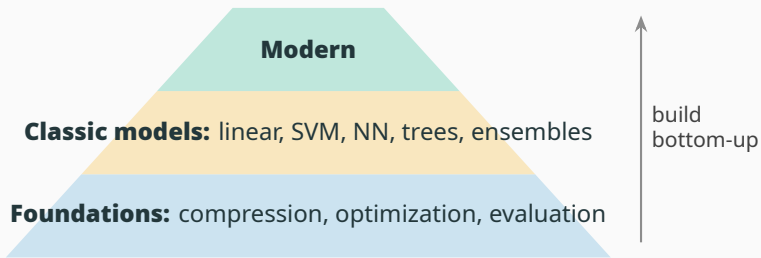
2026

Course Introduction

- **Mário Antunes**
- Associate Professor, UA
- Researcher, Instituto de Telecomunicações
- `mario.antunes@ua.pt`
- Office: 19.2.15 (IT1)
- Research:
 - Machine learning, time series
 - Optimization and knee detection
 - Maintains the Python libraries used in this course



A Fundamentals Course

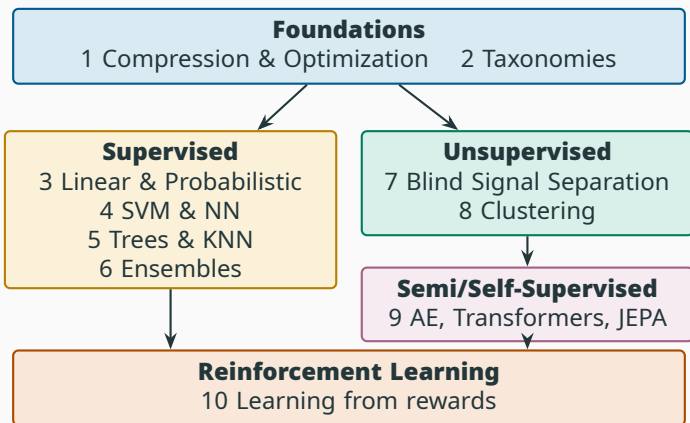


- Understand **why** learning works before using the latest architectures
- Every model in the course rests on the same foundations

Course Topics

1. Learning as Compression and Optimization
2. Learning Taxonomies
3. Supervised Learning: Linear and Probabilistic Models
4. Supervised Learning: SVM and Neural Networks
5. Supervised Learning: Decision Trees and KNN
6. Supervised Learning: Ensemble Models
7. Unsupervised Learning: Blind Signal Separation
8. Unsupervised Learning: Soft/Hard Clustering
9. Semi-Supervised and Self-Supervised Learning
10. Reinforcement Learning

How the Topics Connect



Schedule I: Classes 01–07

#	TP1	TP2	Topic	Milestone
01	17 Sep	18 Sep	T1 + T2: Compression, Pitfalls, Taxonomies	P1 released
02	24 Sep	25 Sep	T1: Blind vs. Gradient Optimization	
03	01 Oct	02 Oct	T3: Linear Models	
04	08 Oct	09 Oct	T3: Probabilistic Models	
05	15 Oct	16 Oct	T4: SVM and Neural Networks	
06	22 Oct	23 Oct	T5: Decision Trees and KNN	P1 checkpoint
07	29 Oct	30 Oct	T6: Ensemble Models	

- T = Topic, P = Project; TP1 on Thursdays, TP2 on Fridays

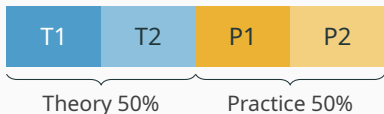
Schedule II: Classes 08-14

#	TP1	TP2	Topic	Milestone
08	05 Nov	06 Nov	Test 1 (T1-T6) + P1 synthesis	P1 due
09	12 Nov	13 Nov	T7: Blind Signal Separation	P2 released
10	19 Nov	20 Nov	T8: Soft/Hard Clustering	
11	26 Nov	27 Nov	T9: AutoEncoders, Semi-Supervised	
12	03 Dec	04 Dec	T9: Transformers and JEPA	
13	10 Dec	11 Dec	T10: Reinforcement Learning	P2 clinic
14	17 Dec	18 Dec	Test 2 (T7-T10) + P2 presentations	P2 due

- Classes end with the tests; projects are defended in Class 14

Evaluation Scheme

Discrete evaluation (recommended)

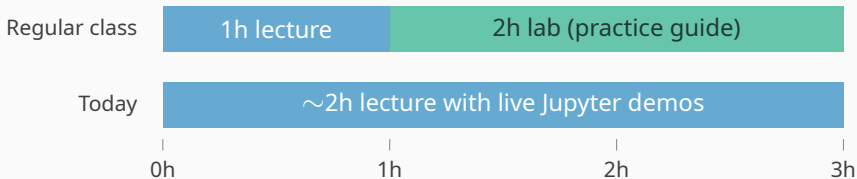


Final evaluation



- **Test 1** (25%): Class 08, Topics 1–6 · **Test 2** (25%): Class 14, Topics 7–10
- **Project 1** (25%): released Class 03, due Class 08
- **Project 2** (25%): released Class 09, due Class 14

Class Format



- Lectures introduce the ideas; labs turn them into code
- Today there is no lab guide: the demos run **during** the lecture

Setting Up the Environment

```
git clone https://github.com/detiuaveiro/faa.git && cd faa
export KERAS_BACKEND=jax      # add to ~/.bashrc
make venv                     # or: python3 -m venv venv && venv/bin/pip
source venv/bin/activate
jupyter lab notebooks/01-foundations/
```

- Today's demos in notebooks/01-foundations/:
 1. 01_learning_as_compression.ipynb
 2. 02_evaluation_validation.ipynb
 3. 03_pitfalls.ipynb
 4. 04_taxonomies.ipynb

The Software Stack

Presentation: JupyterLab

Data & plots: polars matplotlib seaborn

Models: scikit-learn Keras 3 (JAX backend) joblib

Acceleration: NumPy (OpenBLAS) JAX

Group libraries: pyBlindOpt, EmptySpaceSearch,
kneeliverse, pyUTSAlgorithms, pyNNMF

What is Learning?

The Core Question



what did it *learn*?

- What does it mean for a machine to **learn**?
- It receives examples from the past
- It must act well on situations it has **never seen**
- Today: why this is possible at all, and how it fails

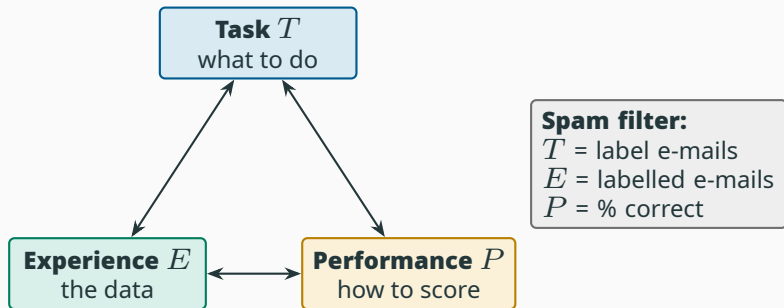
A Classic Definition

*"A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ."*

— Tom Mitchell, Machine Learning, 1997

- Learning is defined by **improvement**
- Improvement needs a **measure**
- Improvement comes from **experience** (data)

Task, Experience, Performance

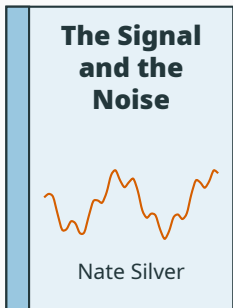


- Every model in this course is described by these three ingredients

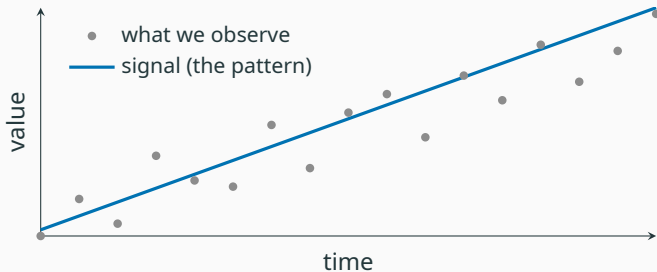
The Signal and the Noise

Why So Many Predictions Fail

- *The Signal and the Noise: Why So Many Predictions Fail — but Some Don't*
- Nate Silver, 2012
- Case studies: weather, elections, baseball, poker, earthquakes, the 2008 financial crisis
- The practical art of building models with **probability and statistics**

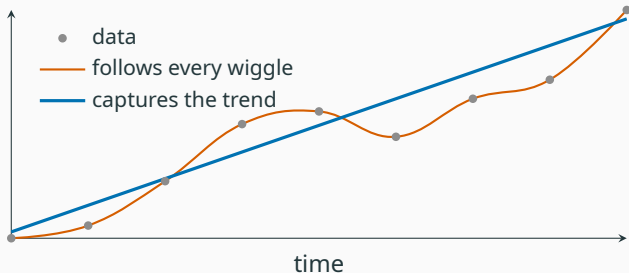


Signal and Noise



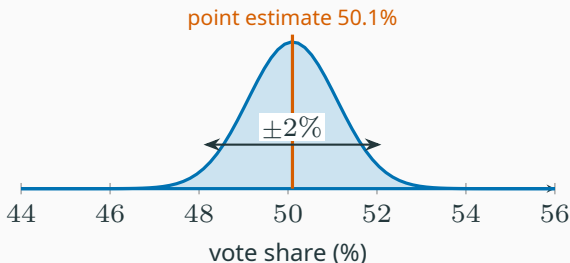
- **Signal:** the regularity that repeats and lets us predict
- **Noise:** random variation that does not repeat
- Every dataset mixes both, and they are not labelled

Mistaking Noise for Signal



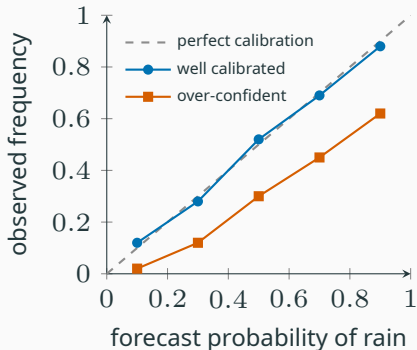
- The world has far more **noise** than **signal**
- A story that explains *every* past wiggle usually predicts the future badly
- More data does not help if we cannot tell the two apart

Uncertainty: Ranges, Not Single Numbers



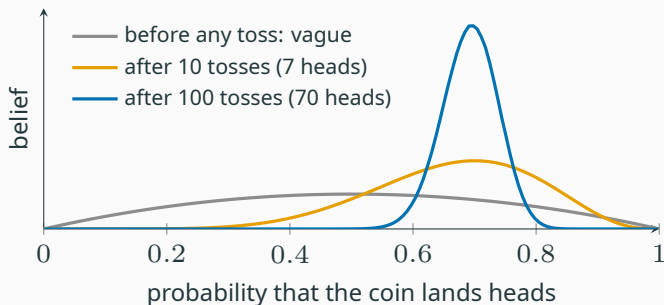
- "Candidate A has an edge" is vague
- "A wins with **probability 83%**, vote share **50.1% $\pm$ 2%**" is useful
- A good model states **how sure** it is, not only its best guess

Calibration



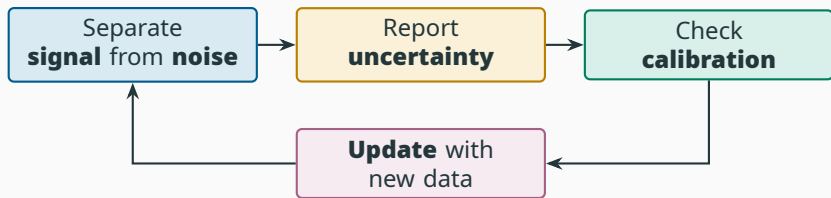
- Weather forecasters: when they say **70% rain**, it should rain on about **70%** of those days
- Calibration checks whether stated probabilities **match reality**

Updating Beliefs with Data



- Start with a belief, observe data, **update** the belief (Bayes' rule, Topic 3)
- With more data the belief becomes **sharper**
- Learning is gradual, not a single perfect model

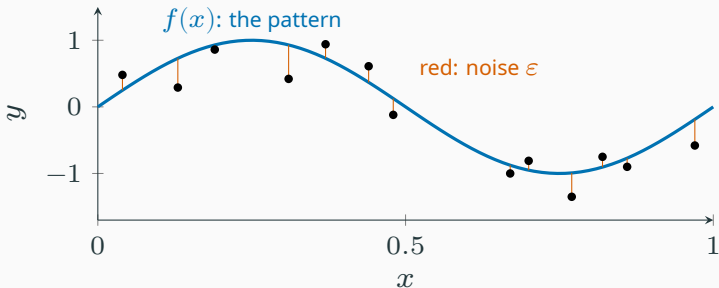
Lessons for Machine Learning



- These ideas return throughout the course:
 - Compression separates signal from noise (today)
 - Statistical validation measures uncertainty (today)
 - Probabilistic models and Bayes' rule (Topic 3)

Pattern, Noise, and Memorization

Data = Pattern + Noise



$$y = f(x) + \epsilon$$

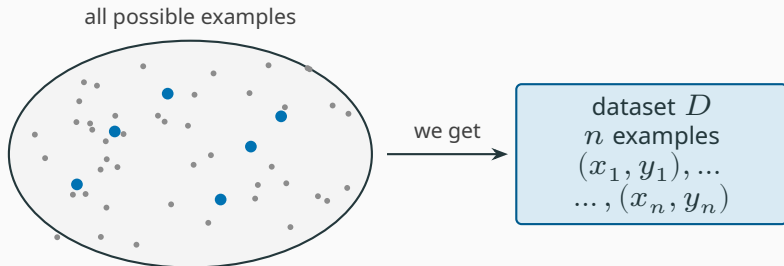
- x : the input we observe; y : the output we want to predict
- $f(x)$: the unknown pattern; ϵ (epsilon): the noise, a random error

Reading the Formula

$$\underbrace{y}_{\text{what we measure}} = \underbrace{f(x)}_{\text{the rule we want}} + \underbrace{\varepsilon}_{\text{random error}}$$

- **Example:** house price
 - x = area of the house
 - $f(x)$ = the “fair” price for that area
 - ε = everything else: the seller’s mood, a nice view, luck
- We **never** see f directly, only pairs (x, y)
- Learning means building an estimate $\hat{f}$ (“f-hat”) of f

We Only See a Finite Sample



- $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$: n observed pairs; the index i numbers them
- The goal is **not** to reproduce D
- The goal is to predict well on **new** x from the same process

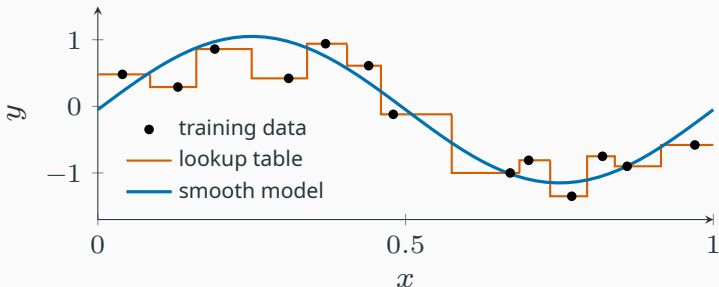
Memorization: the Lookup Table

x	y
0.04	0.48
0.13	0.29
0.19	0.86
...	...
0.97	-0.58

new input $x = 0.15$
copy the answer of the
closest stored x

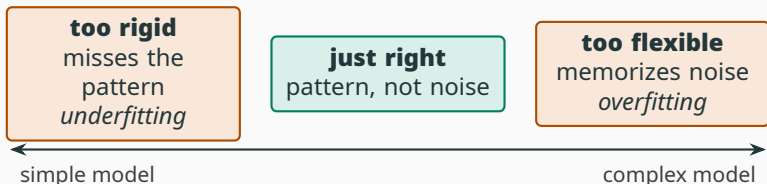
- Stores **every** training example
- Perfect on data it has seen: training error = 0
- Stores the noise ε as faithfully as the pattern f

Generalization



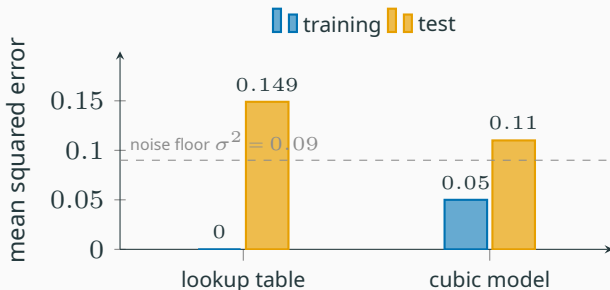
- **Generalization:** performing well on data **not** seen during training
- The lookup table jumps with every noisy point; the smooth model captures the pattern

The Dilemma: Too Flexible or Too Rigid



- We need a **principled** way to prefer simple explanations
- Answer today: **compression** (and later, validation)
- Under- and overfitting are revisited in detail as a pitfall

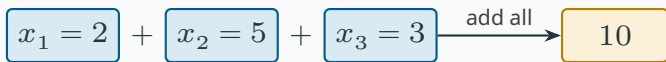
Live Demo D1: Memorization vs. Generalization



- Notebook `01_learning_as_compression.ipynb`, section **D1**: 50 noisy points
- Lookup table: **100 numbers** stored; cubic polynomial: **4 numbers**
- Which one has lower training error? Lower test error? Why?

A Small Math Toolbox

Summation: Adding Many Things

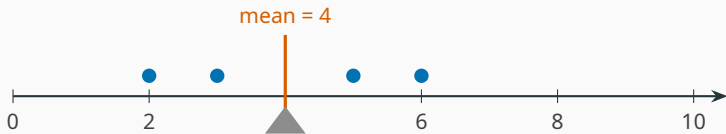


$i = 1, 2, 3$ (the index counts the items)

$$\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

- $\sum$ (capital sigma) means **“add up”**
- $i = 1$ below: start at the first item; n on top: stop at the last item
- Example with $n = 3$: $\sum_{i=1}^3 x_i = 2 + 5 + 3 = 10$

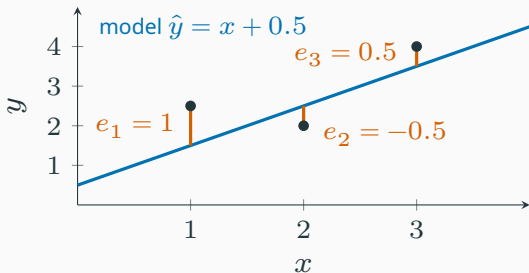
The Mean: a Typical Value



$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- $\bar{x}$ ("x-bar"): the average; $\frac{1}{n}$: divide the total by how many items there are
- Example: values 2, 5, 3, 6 $\Rightarrow \bar{x} = \frac{2+5+3+6}{4} = \frac{16}{4} = 4$
- The mean is the **balance point** of the values

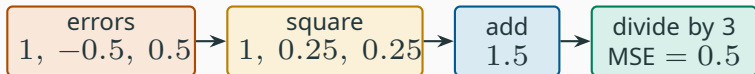
Errors: How Wrong Is a Prediction?



- **Error (residual):** $e_i = y_i - \hat{y}_i$ = true value — prediction
- $\hat{y}_i$ ("y-hat"): what the model predicts for example i
- Errors can be positive or negative, so we cannot just add them: they cancel out

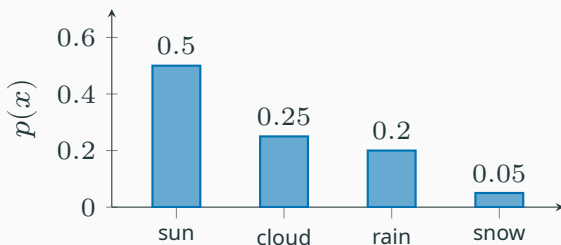
Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



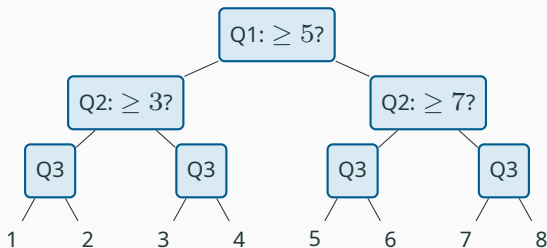
- **Squaring** makes every error positive and punishes large errors more
- $\text{MSE} = 0$ means perfect predictions; larger means worse
- We use MSE throughout the course to measure regression models

Probability: How Likely Is Each Outcome?



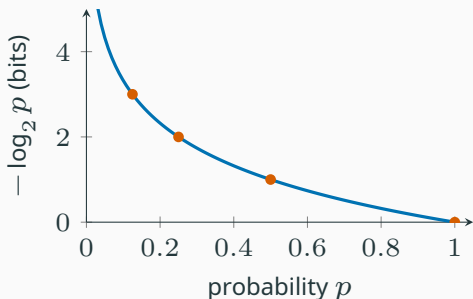
- $p(x)$: the probability of outcome x , a number between **0** (never) and **1** (always)
- All probabilities add up to one:
$$\sum_x p(x) = 0.5 + 0.25 + 0.2 + 0.05 = 1$$
- Probability 0.25 means “happens about 1 time in 4”

Logarithm Base 2: Counting Yes/No Questions



- Guess a number from 1 to 8 with yes/no questions: **3 questions** always suffice
- $\log_2 8 = 3$ because $2^3 = 8$; in general $\log_2 N =$ questions needed for N equally likely options
- One yes/no answer = **1 bit** of information

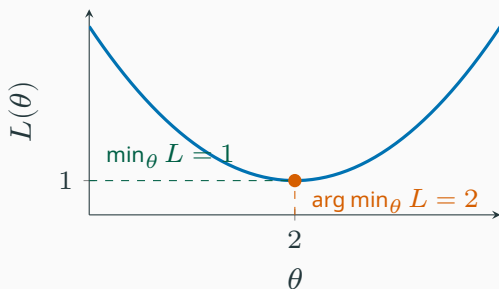
Logarithms of Probabilities



p	$-\log_2 p$
1	0 bits
1/2	1 bit
1/4	2 bits
1/8	3 bits

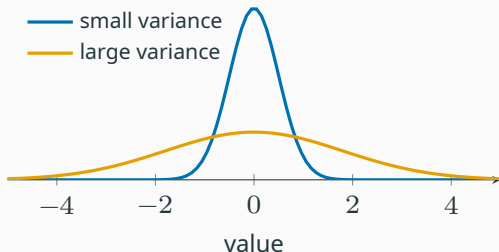
- For $p < 1$, $\log_2 p$ is negative, so we use $-\log_2 p$ to get a positive number
- **Certain** events ($p = 1$) carry 0 bits; **rare** events carry many bits

argmin: Where Is the Minimum?



- $L(\theta)$: a function of a parameter θ (theta), e.g. the error of a model
- $\min_{\theta} L(\theta)$: the **smallest value** of the function (here 1)
- $\arg \min_{\theta} L(\theta)$: the **parameter that achieves it** (here $\theta = 2$)

Variance: How Spread Out Are the Values?

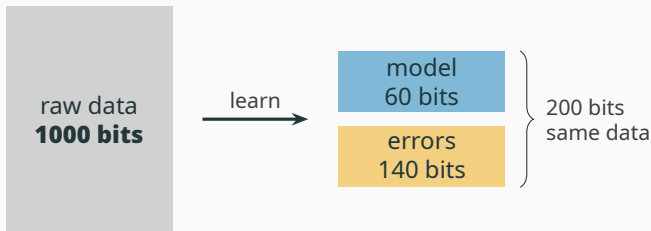


$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad s = \sqrt{s^2}$$

- Variance s^2 : average squared distance to the mean; standard deviation s : its square root
- Example: 2, 5, 3, 6 with $\bar{x} = 4$: $\frac{(4+1+1+4)}{3} = 3.33$, so $s \approx 1.83$

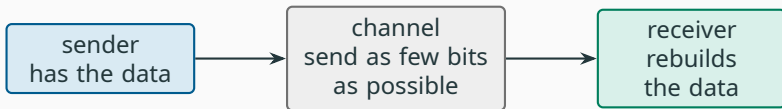
Learning as Compression

To Learn Is to Compress



- If a short model plus its errors **reproduces** the data, the model found regularities
- The model is a **compressed** description of the data
- A model that is as big as the data (a lookup table) has **learned nothing**

Compression as Communication



"temperature rises 2 °C per hour,
plus these small corrections"
is much shorter than sending every reading

- To send less, the sender must **understand** the data
- Understanding = finding the rule that generates most of it
- Only the part the rule cannot explain (the noise) must be sent in full

Pure Noise Cannot Be Compressed

ABABABABABABABABABABABAB

→ "AB × 12"

THE CAT SAT ON THE MAT.

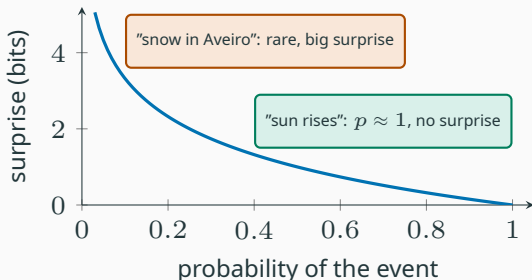
→ some savings

Q7#pZ2!mK9vX@4rT&1wL8

→ no shorter description

- **Structure** can be described briefly; **randomness** cannot
- If data were pure noise, there would be **nothing to learn**
- Learning is possible **because** real data contain regularities

Surprise: Information of One Event



$$h(x) = -\log_2 p(x)$$

- $h(x)$: the information (surprise) of observing outcome x , measured in **bits**
- Likely events tell us little; unlikely events tell us a lot

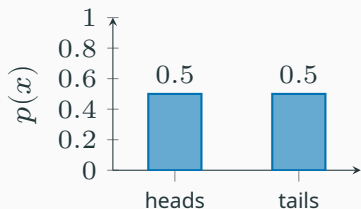
Shannon Entropy

$$H(X) = - \sum_x p(x) \log_2 p(x)$$



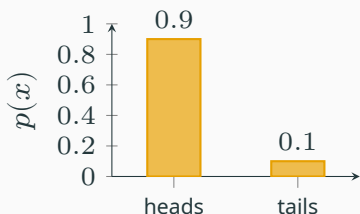
- X : a random variable, e.g. “the result of a coin toss”
- $H(X)$: the **average surprise**, in bits per outcome (Claude Shannon, 1948)
- High entropy = unpredictable; low entropy = predictable

Worked Example: a Fair Coin



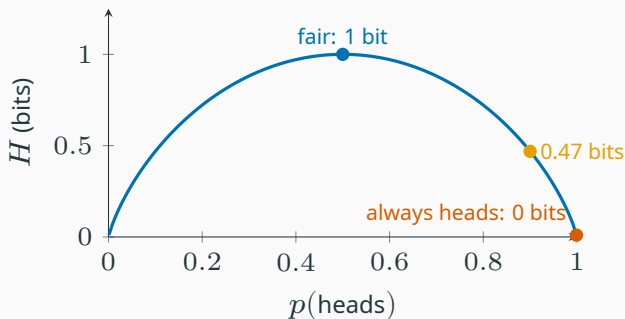
1. Probabilities: $p(\text{heads}) = 0.5, p(\text{tails}) = 0.5$
 2. Surprise of each: $-\log_2 0.5 = 1 \text{ bit}$
 3. Weight and add: $H = 0.5 \times 1 + 0.5 \times 1 = 1 \text{ bit}$
- A fair coin needs one full bit per toss: nothing can be predicted

Worked Example: a Biased Coin



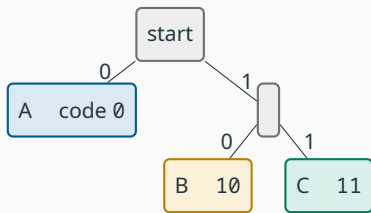
1. Probabilities: $p(\text{heads}) = 0.9$, $p(\text{tails}) = 0.1$
 2. Surprise: $-\log_2 0.9 \approx 0.152$ bits; $-\log_2 0.1 \approx 3.32$ bits
 3. Weight and add: $H = 0.9 \times 0.152 + 0.1 \times 3.32 \approx 0.137 + 0.332 = 0.47$ **bits**
- Mostly predictable: on average less than half a bit per toss

Entropy Across All Coins



- Entropy is **maximal** when all outcomes are equally likely
- Entropy is **zero** when the outcome is certain

Entropy Is the Limit of Compression



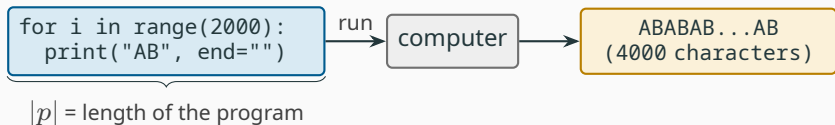
$$p(A) = 0.5, p(B) = 0.25, p(C) = 0.25$$

average length:
 $0.5 \times 1 + 0.25 \times 2 + 0.25 \times 2 = 1.5 \text{ bits}$

entropy: $H = 1.5 \text{ bits}$

- Give **short codes to frequent symbols** and long codes to rare ones
- No lossless code can use fewer bits on average than H (Shannon's source coding theorem)

Kolmogorov Complexity



$$K(x) = \min_p \{ |p| : U(p) = x \}$$

- $K(x)$: length of the **shortest program** p that prints the string x
- $U(p) = x$: "running program p on a computer U outputs x "; $|p|$: length of p
- The ultimate measure of how much structure a string contains (Kolmogorov, 1965)

Two Strings, Two Shortest Programs

ABAB . . . AB (4000 chars)

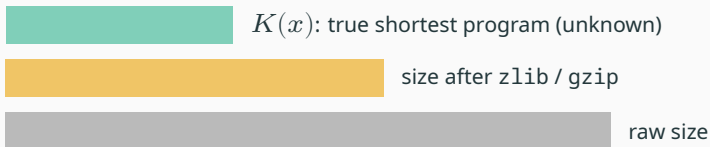
 program: ≈ 40 characters

4000 random characters

program: `print("...all 4000 characters...")` ≈ 4000 characters

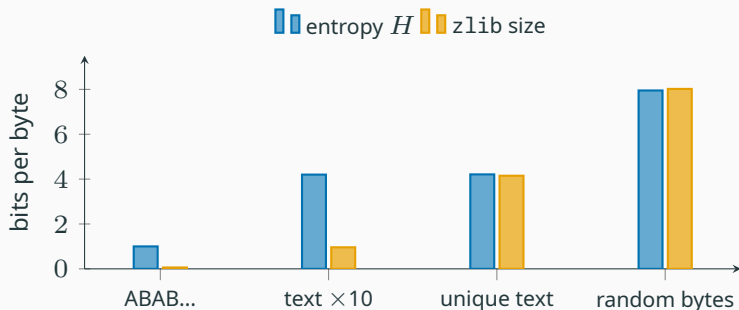
- Regular string: K is **tiny** compared with its length
- Random string: $K \approx$ its length, it is **incompressible**
- Learning looks for the short program behind the data

Kolmogorov Complexity in Practice



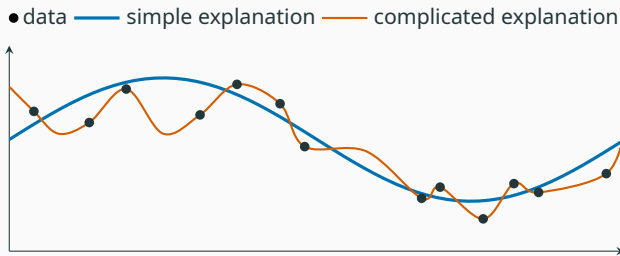
- $K(x)$ **cannot be computed**: no algorithm finds the shortest program for every string
- Real compressors give an **upper bound**: $K(x) \leq$ compressed size
- Machine learning models play the same role: practical, imperfect compressors

Live Demo D2: Entropy and Compression



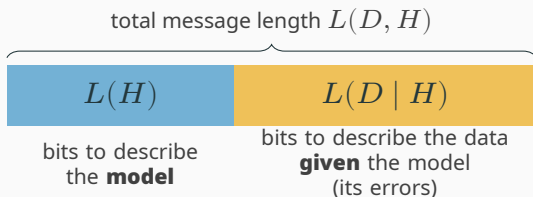
- Notebook `01_learning_as_compression.ipynb`, section **D2**
- Why does zlib beat the entropy on ABAB...? Why does random data **grow**?

Occam's Razor



- *"Entities should not be multiplied beyond necessity"*
(William of Ockham, 14th century)
- Among explanations that fit the data similarly well,
prefer the simplest
- But how do we measure "simple" and "fits well" on the same scale?

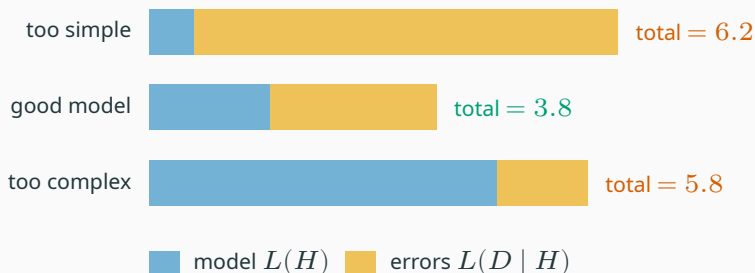
Minimum Description Length (MDL)



$$L(D, H) = L(H) + L(D | H)$$

- H : a hypothesis (a model); D : the data; $L(\cdot)$: length in bits
- $D | H$ reads “ D given H ”: what is left to describe once the model is known
- **MDL principle** (Rissanen, 1978): choose the model with the **shortest total** description

The MDL Trade-Off



- **Too simple:** tiny model, but the errors take many bits
- **Too complex:** tiny errors, but the model itself is huge (it encodes the noise)
- The minimum total balances **fit** and **complexity** automatically

MDL in Practice: BIC

$$L(D, H) \approx \underbrace{\frac{k}{2} \log_2 n}_{\text{model cost } L(H)} + \underbrace{\frac{n}{2} \log_2 \text{MSE}}_{\text{error cost } L(D|H)}$$

k : number of parameters
each costs $\frac{1}{2} \log_2 n$ bits
grows with model size

n : number of examples
smaller MSE $\Rightarrow$ fewer bits
shrinks as the fit improves

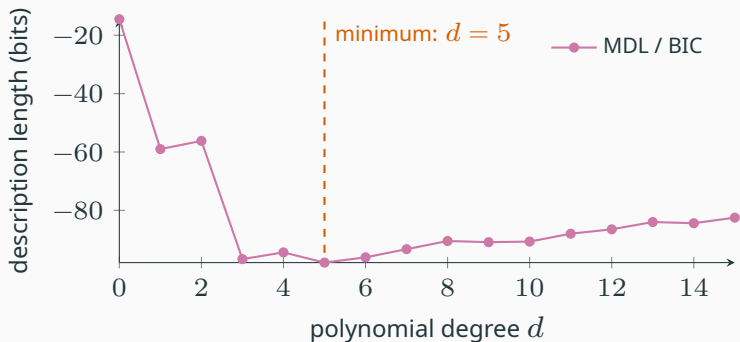
- This approximation is the **Bayesian Information Criterion (BIC)**, written in bits
- It uses **only the training data** to choose model complexity
- Constants are dropped, so values can be negative: only **differences** between models matter

Worked Example: Choosing a Polynomial Degree

Model	k	model cost $\frac{k}{2} \times 5.64$	MSE	error cost $25 \times \log_2 \text{MSE}$	total
degree 1 (line)	2	5.6	0.167	-64.6	-59.0
degree 5	6	16.9	0.042	-114.8	- 97.9
degree 15	16	45.2	0.029	-127.6	-82.5

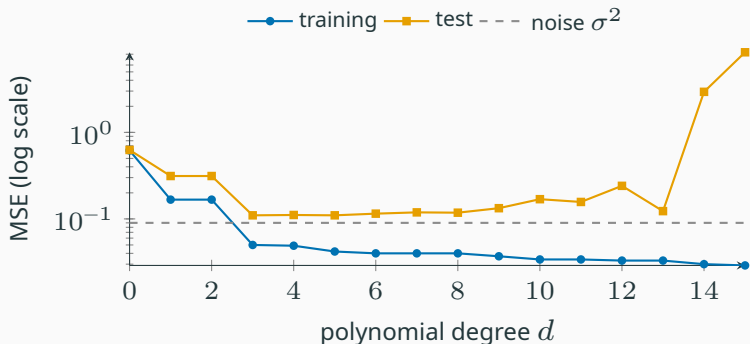
- Data from demo D1: $n = 50$ points, so $\log_2 50 \approx 5.64$
- Going from degree 5 to 15: errors save **12.8 bits**, the model costs **28.3 bits** more
- **Degree 5 wins**: the lowest total description length

The MDL Curve



- The curve drops while extra parameters capture **pattern**
- It rises again once extra parameters only encode **noise**

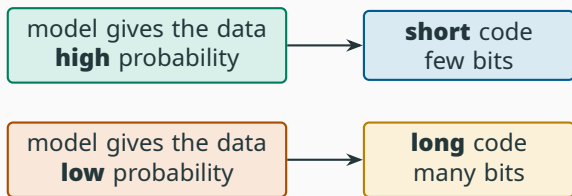
Live Demo D3: Model Selection as Compression



- Notebook 01_learning_as_compression.ipynb, section **D3**
- Training error **always** decreases; test error explodes for $d \geq 14$
- MDL picked $d = 5$ **without** looking at the test data

Compression in Modern Machine Learning

Likelihood Is a Code Length



$$L(D \mid H) = -\log_2 P(D \mid \theta)$$

- $P(D \mid \theta)$: the probability the model with parameters θ assigns to the data (the **likelihood**)
- Maximizing likelihood = **minimizing code length**: the same objective

Regularization Is a Description Length

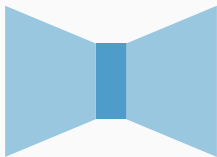
$$\overbrace{\mathcal{L}_{data}(D; \theta) + \lambda \Omega(\theta)}^{\text{total training loss } \mathcal{L}_{total}(\theta)}$$

$\mathcal{L}_{data}(D; \theta): \text{fit errors}$	$\lambda \Omega(\theta)$
$\approx L(D H)$	$\approx L(H)$

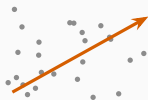
$$\mathcal{L}_{total}(\theta) = \mathcal{L}_{data}(D; \theta) + \lambda \Omega(\theta)$$

- $\Omega(\theta)$: a **penalty** for complex models, e.g. the sum of squared weights
- λ (lambda): how much we care about simplicity versus fit
- The same two-part trade-off as MDL, in the language of optimization (Topic 3)

Compression Across the Course



AutoEncoder:
bottleneck (Topic 9)



PCA: keep the main
direction (Topic 7)

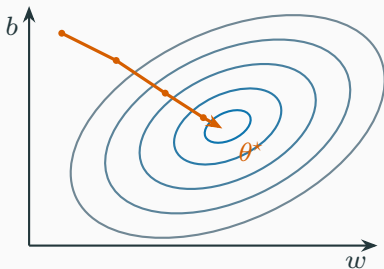


Clustering: a few
prototypes (Topic 8)

- The same idea appears again and again: **summarize the data with less**

From Compression to Optimization

Finding the Model Is an Optimization Problem



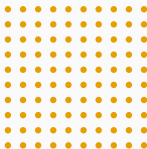
$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

- MDL tells us **what** a good model is; optimization tells us **how** to find its parameters
- Contours: lines of equal loss; the red path walks downhill to the minimum θ^*

Why Not Try Every Value?



1 parameter: 10 tries

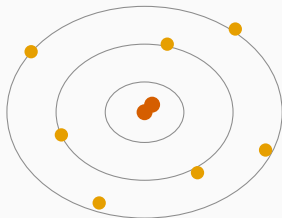


2 parameters: 100 tries

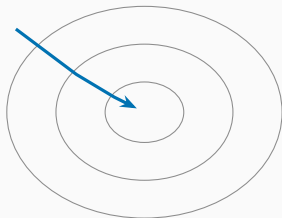
1000 parameters:
 10^{1000} tries
impossible

- A grid with 10 values per parameter needs 10^k evaluations for k parameters
- Real models have thousands to billions of parameters
- We need **smarter search**

Next Class: Two Ways to Search



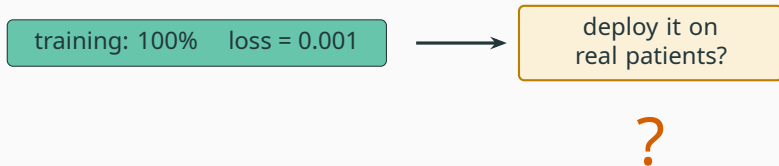
Blind optimization
a population of guesses,
no derivatives needed



Gradient-based
follow the slope downhill,
derivatives by hand and JAX

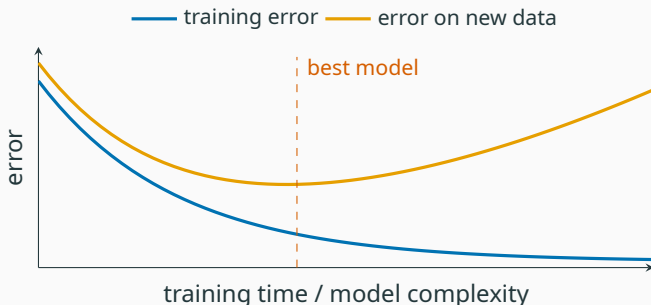
Evaluation and Statistical Validation

Training Finished. Are We Done?



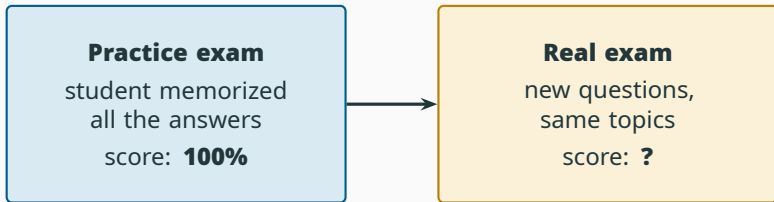
- The optimizer stopped: it found parameters with **low training loss**
- That only says the model fits the **examples it was given**
- It says **nothing yet** about the cases the model will actually face

Training Loss Is Not the Goal



- Training error keeps going **down**
- Error on new data goes down, then **up**: the model starts memorizing noise
- We need an **independent** measurement to see the second curve

The Exam Analogy



- A perfect score on questions you have already seen proves **memory**, not understanding
- To grade understanding, the teacher uses **questions the student never saw**
- Evaluation of a model follows exactly the same logic

Why Measure Performance at All?

Is it good enough?
deploy or not

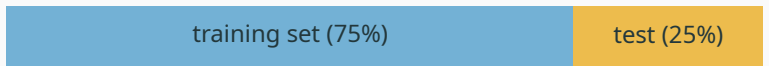
**Is it better than
what we have?**
baseline, current system

Which model?
which settings?

How sure are we?
could it be luck?

- Every decision about a model needs an **honest estimate** of its future performance
- Mistakes are costly: a missed diagnosis, a fraud not detected, a wrong forecast

Hold Out Data



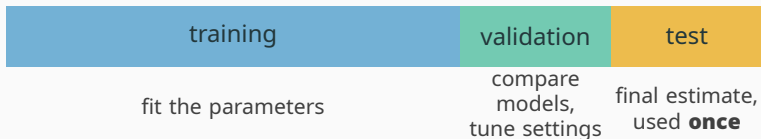
the model learns from these

hidden until the end

Decision tree (demo D4a):
training accuracy **1.000**
test accuracy **0.923**

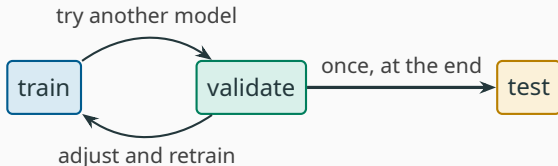
- Split the data **before** training
- Accuracy on training data measures **memorization**; on the test set, **generalization**

Training, Validation, and Test Sets



- **Training set:** the model learns its parameters here
- **Validation set:** we make choices here (which model, which complexity)
- **Test set:** a final, unbiased estimate of performance on new data

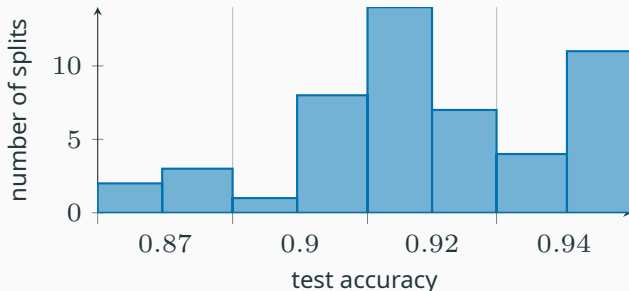
The Test Set Is Used Once



looking at the test score and then changing the model turns the test set into a validation set

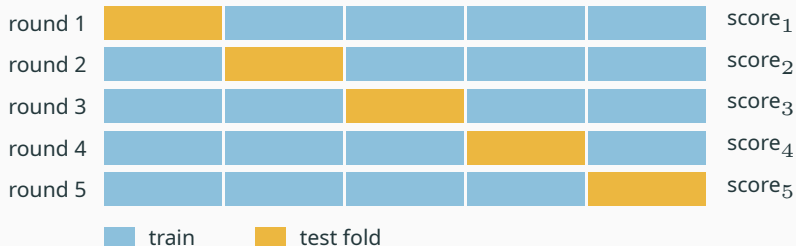
- Every decision based on the test set makes its estimate **optimistic**

One Split Is Not Enough



- Same model, same data, **50 different random splits** (demo D4b)
- Test accuracy ranged from **0.874** to **0.965**
- A single number like “92.3%” hides this variation

k-Fold Cross-Validation



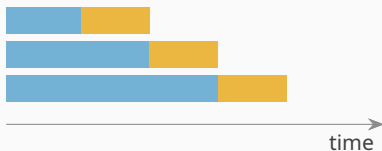
- Split the data into k parts (**folds**); here $k = 5$
- Each round trains on $k - 1$ folds and tests on the remaining one
- Every example is tested **exactly once**; we get k scores instead of one

Cross-Validation Variants

Stratified: same class mix per fold

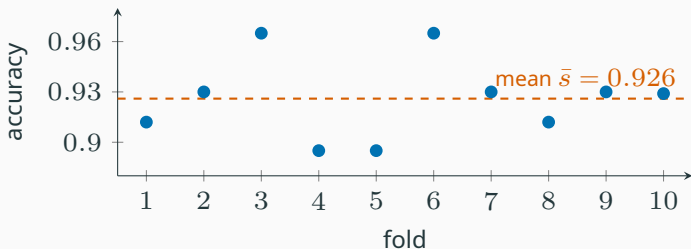


Temporal: never train on the future



- **Stratified:** keep class proportions equal in every fold
- **Grouped:** all samples of the same patient/user stay in the same fold
- **Temporal:** for time series, always test on data **after** the training period

Summarizing the Folds



- Decision tree, 10-fold cross-validation (demo D4c): 10 accuracy scores $s_1, \dots, s_{10}$
- **Mean:** $\bar{s} = \frac{1}{10} \sum_{i=1}^{10} s_i = 0.926$
- **Standard deviation:** $s = 0.025$ (the folds disagree by a few points)

Confidence Interval

$$\bar{s} \pm t_{0.975, k-1} \times \frac{s}{\sqrt{k}}$$

$\bar{s}$
best guess
(mean score)

$s/\sqrt{k}$
standard error:
uncertainty
of the mean

$t_{0.975, k-1}$
multiplier for 95%
confidence (≈ 2)

- k : number of folds; s : standard deviation of the fold scores
- **95% confidence interval:** a range that, in repeated experiments, contains the true performance 95% of the time

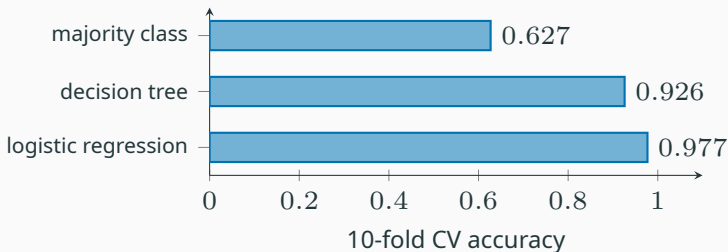
Worked Example: Confidence Interval

1. Mean and standard deviation: $\bar{s} = 0.926$, $s = 0.025$, $k = 10$
2. Standard error: $\frac{0.025}{\sqrt{10}} = \frac{0.025}{3.16} = 0.0079$
3. Multiplier (table value for $k - 1 = 9$): $t_{0.975, 9} = 2.26$
4. Half-width: $2.26 \times 0.0079 = 0.018$



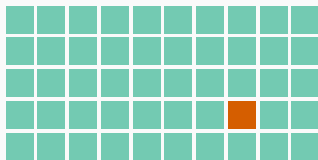
- Report: “accuracy **0.926 ± 0.018** (95% CI, 10-fold CV)”, not just “92.6%”

Always Compare with a Baseline



- **Baseline:** the simplest possible predictor, e.g. “always predict the most common class”
- On this dataset the baseline already scores **62.7%**: “70% accuracy” would be poor
- A model is only useful if it clearly **beats the baseline**

The Accuracy Paradox



50 patients, 1 sick
model: "everyone is healthy"
accuracy = $49/50 = 98\%$
sick patients found = 0

- When one class is rare, accuracy can look excellent for a **useless** model
- Demo D4d: 98% accuracy, balanced accuracy 50%, F1 = 0
- **The metric must match the goal**

The Confusion Matrix

		predicted	
		sick	healthy
actual	sick	TP = 6 true positive	FN = 4 missed
	healthy	FP = 2 false alarm	TN = 88 true negative

- 100 patients, 10 of them sick; the model flags 8 patients as sick
- Each cell counts one kind of **right** or **wrong** decision

Precision, Recall, and F1

$$\text{precision} = \frac{TP}{TP + FP} \quad \text{recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

precision

of those flagged,
how many are sick?

$$\frac{6}{6+2} = 0.75$$

recall

of the sick,
how many
were found?

$$\frac{6}{6+4} = 0.60$$

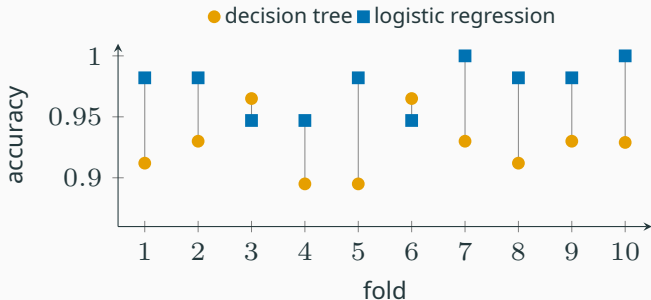
F1

balance of both

$$\frac{2 \times 0.75 \times 0.6}{0.75 + 0.6} = 0.67$$

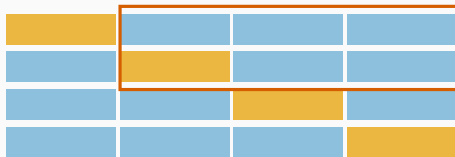
- Screening for a disease: high **recall** matters (do not miss sick patients)
- Spam filter: high **precision** matters (do not hide real e-mails)

Comparing Two Models: Pair the Scores



- Evaluate both models on **the same folds** and look at the difference per fold: $d_i = a_i - b_i$
- Pairing removes the variation caused by “easy” and “hard” folds
- Logistic regression wins in 8 of 10 folds, but the tree wins in folds 3 and 6

Why a Naive t-Test Is Over-Confident



rounds 1 and 2 share
2 of their 3 training folds:
their scores are
not independent

- A standard t-test assumes the k scores are **independent** measurements
- In cross-validation the training sets **overlap**, so the scores are correlated
- The naive test underestimates the variance and finds “significant” differences too easily

The Corrected Resampled t-Test

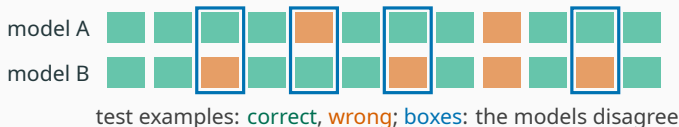
$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{test}}{n_{train}}\right) s_d^2}}$$

- $\bar{d}$: mean of the per-fold differences; s_d^2 : their variance; k : number of scores
- $\frac{n_{test}}{n_{train}}$: extra term that **inflates the variance** to account for overlap (Nadeau & Bengio, 2003)
- Demo D4e, logistic regression vs. decision tree, 5×10 -fold CV, mean difference $+0.052$:

naive paired t-test
 $p = 3.6 \times 10^{-15}$
(far too confident)

corrected test
 $p = 6.0 \times 10^{-5}$
(still significant, honestly)

McNemar's Test: One Test Set, Two Models



- Sometimes cross-validation is too expensive: we train once and have **one test set**
- Both models predict the **same** test examples
- Only the examples where they **disagree** tell us which model is better

McNemar's Contingency Table

		decision tree	
		correct	wrong
logistic regression	correct	$a = 131$ both right	$b = 10$ only LR right
	wrong	$c = 1$ only tree right	$d = 1$ both wrong

- Demo D4f: the 143 test examples of the breast cancer dataset
- a and d say **nothing** about which model is better: both behave the same
- If the models were equally good, the disagreements would split evenly: $b \approx c$

McNemar's Statistic

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}$$

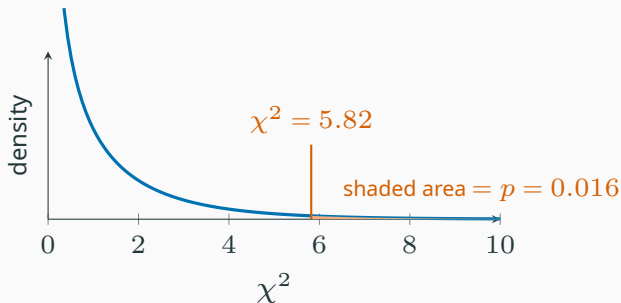
$|b - c|$
how unbalanced
they are

-1
small-sample
correction

$b + c$
total number of
disagreements

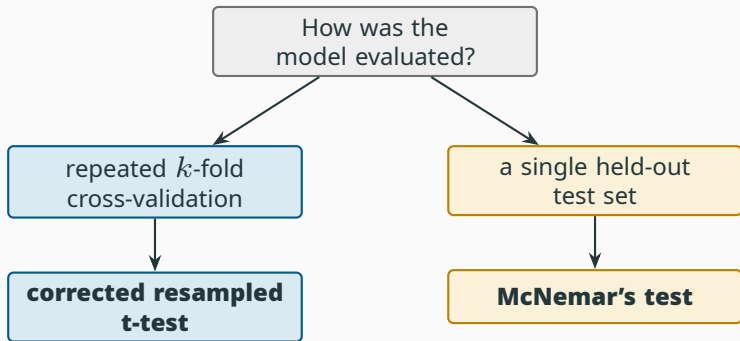
- χ^2 ("chi-squared"): large when one model wins most disagreements
- If both models are equally good, χ^2 follows a **chi-squared distribution with 1 degree of freedom**
- **p-value:** probability of a χ^2 this large (or larger) if the models were equally good
- If $b + c$ is small (below about 25), use the exact **binomial** version

Worked Example: McNemar



1. Disagreements: $b = 10$, $c = 1$, so $|b - c| = 9$ and $b + c = 11$
2. Statistic: $\chi^2 = \frac{(9-1)^2}{11} = \frac{64}{11} = 5.82$
3. p-value = $0.016 < 0.05$ (exact binomial: 0.012):
logistic regression is **significantly** better

Which Test When?



- Both answer: "is the difference **real**, or could it be **luck**?"
- A significant difference must also be **large enough to matter** in practice

Live Demo D4: Evaluation and Validation

D4a training
vs. test

D4b 50 random
splits

D4c 10-fold CV
and CI

D4d baselines
and metrics

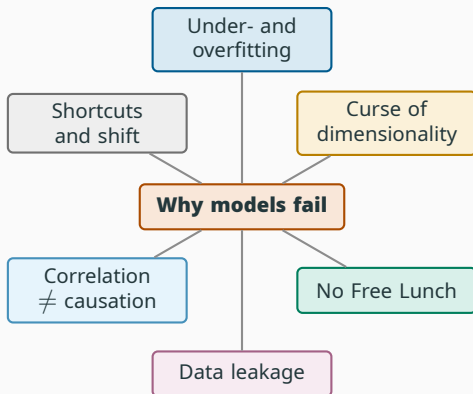
D4e corrected
t-test

D4f McNemar's
test

- Notebook `02_evaluation_validation.ipynb`, breast cancer dataset (569 patients, 30 features)
- Evaluation will be **revisited in every topic**: regression metrics, clustering without labels, time series

Limitations and Pitfalls

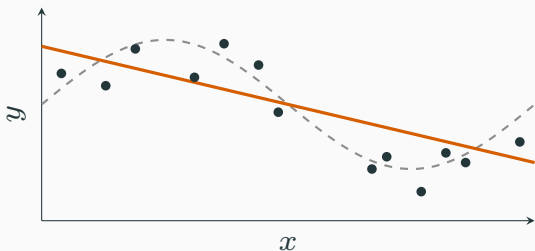
A Map of the Pitfalls



- A model can score well in the lab and **still fail** in the real world
- Knowing these pitfalls is part of doing machine learning **correctly**

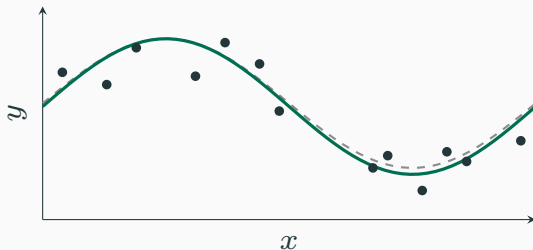
Under- and Overfitting

Underfitting: Too Simple



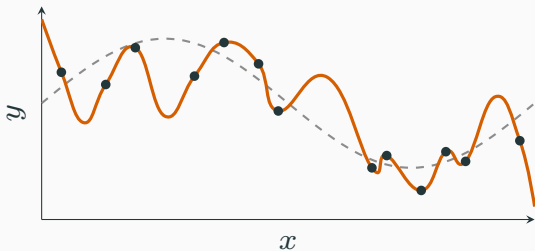
- A straight line cannot follow a wave: the model is **too rigid**
- High error on training data **and** on new data
- The model misses real **pattern** (it compresses too much)

A Good Fit



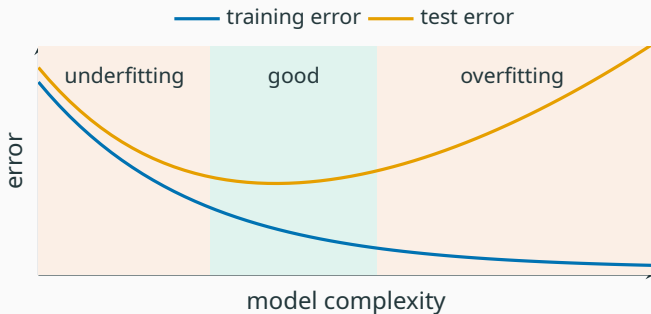
- The model follows the **pattern** and ignores the **noise**
- Small training error, and similar error on new data
- This is the goal: the shortest description that still explains the data

Overfitting: Too Flexible



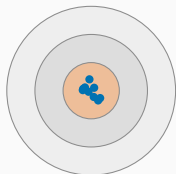
- The curve passes through **every** point, including the noise
- Training error near zero, but **large** error on new data
- The model memorized the sample instead of learning the pattern

Error Versus Model Complexity

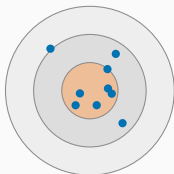


- **Training error** always decreases as the model becomes more complex
- **Test error** is U-shaped: the best model is in the middle

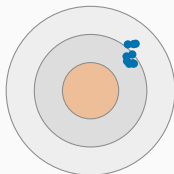
Bias and Variance: the Dartboard



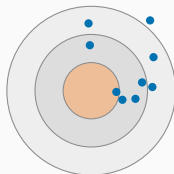
low bias,
low variance



low bias,
high variance



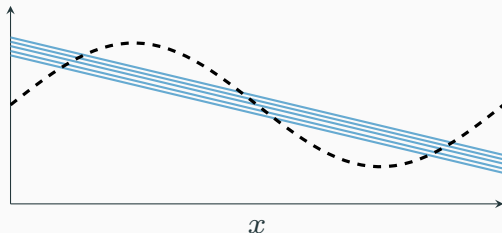
high bias,
low variance



high bias,
high variance

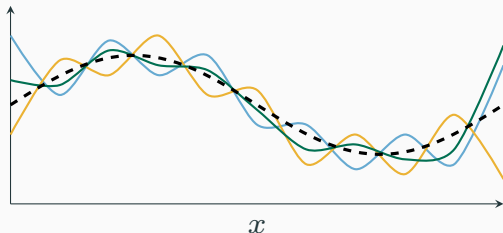
- Centre of the target = the true pattern; each dart = a model trained on a **different** sample
- **Bias**: the darts land off-centre **systematically**
- **Variance**: the darts are **scattered**

Bias: Wrong in the Same Way



- Five straight lines, each fitted to a **different** noisy sample (blue); dashed: the truth
- The lines agree with each other, but all miss the wave in the same way
- **Bias** = the error that remains even with the **average** of many models

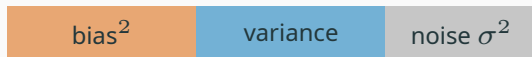
Variance: Different Every Time



- Three very flexible models, each trained on a **different** noisy sample
- On average they follow the wave, but each one wiggles **differently**
- **Variance** = how much the model changes when the training sample changes

The Bias-Variance Decomposition

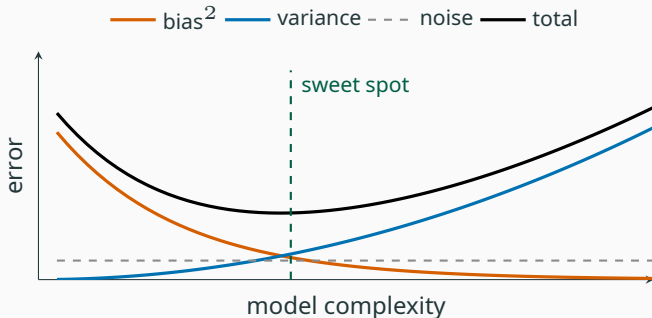
$$\underbrace{\mathbb{E}[(y - \hat{f}(x))^2]}_{\text{expected test error}} = \underbrace{\text{bias}^2}_{\text{too rigid}} + \underbrace{\text{variance}}_{\text{too sensitive}} + \underbrace{\sigma^2}_{\text{noise}}$$



cannot be reduced

- $\mathbb{E}[\cdot]$: the **average** over many possible training sets
- **bias** = (average prediction — truth); **variance** = spread of predictions around their average
- The noise σ^2 sets a floor that **no model** can beat

The Bias-Variance Trade-Off



- Making the model more complex **lowers bias** but **raises variance**
- The best complexity **minimizes the sum**

What to Do About It

Underfitting (high bias)

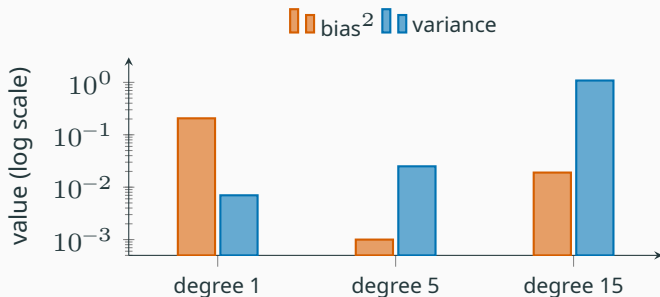
- a more flexible model
- better features
- less regularization
- train longer

Overfitting (high variance)

- more training data
- a simpler model
- more regularization
- ensembles (Topic 6)

- Diagnose first: compare training error and validation error
- Both high $\Rightarrow$ underfitting
- Training low, validation high $\Rightarrow$ overfitting

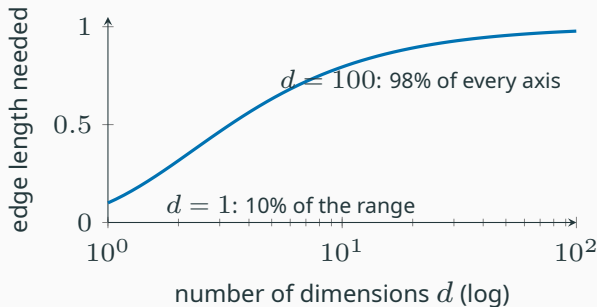
Live Demo D5: Bias and Variance



- Notebook 03_pitfalls.ipynb, section **D5**: 30 training sets, each with fresh noise
- Degree 1: **bias** dominates (0.206)
- Degree 15: **variance** explodes (1.084)
- Degree 5: both small, the best compromise

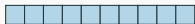
The Curse of Dimensionality

Intuition Breaks in High Dimensions

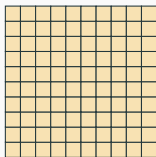


- To collect the **10% nearest** examples in a unit cube, the neighbourhood edge must be $0.1^{1/d}$
- In high dimensions a “local” neighbourhood covers almost the **whole range** of every feature
- “Near” stops meaning “similar”

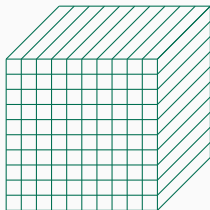
Covering the Space Needs Exponential Data



$d = 1$: 10 cells



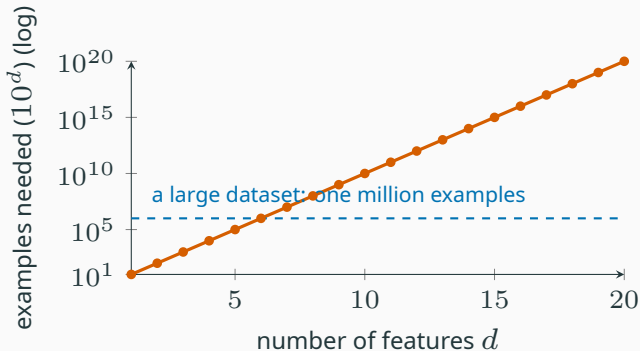
$d = 2$: 100 cells



$d = 3$: 1000 cells

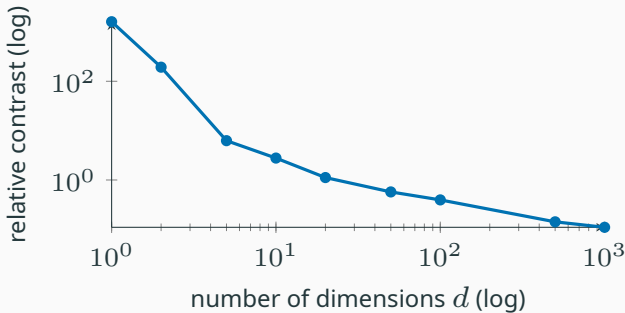
- Split each feature into 10 intervals: the number of cells is 10^d
- To have **one** example per cell we need 10^d examples
- With 20 features: 10^{20} examples, far more than any dataset

Data Needed Grows Exponentially



- A straight line on a log scale means **exponential** growth
- Beyond 6 features, one million examples can no longer fill the grid
- Real data survive only because they live near **lower-dimensional structure**

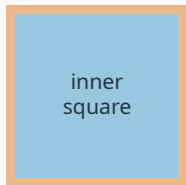
Distances Concentrate



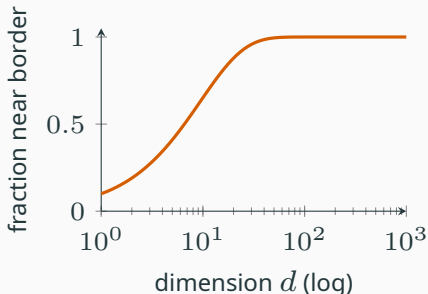
$$\text{relative contrast} = \frac{d_{\max} - d_{\min}}{d_{\min}}$$

- $d_{\min}, d_{\max}$: distance from a query to its **nearest** and **farthest** of 500 random points
- In 1000 dimensions the farthest point is only **11%** farther than the nearest

Almost All Volume Is Near the Border



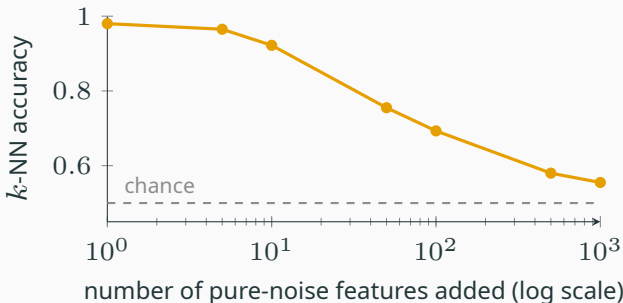
red: within 5%
of the border



$$\text{fraction within 5\% of the border} = 1 - 0.9^d$$

- The inner cube has side 0.9, so its volume is 0.9^d ; the rest is the border region
- $d = 2$: 19% of the volume; $d = 50$: **99.5%**
- Almost every point is an “edge case”

Irrelevant Features Hurt



- 2 informative features + an increasing number of **random** features
- The signal is still there, but distances are dominated by noise
- More features is **not** automatically better

Live Demo D6: Curse of Dimensionality

distance contrast

500 random points
 $d = 1 \dots 1000$

volume near border

$$1 - 0.9^d$$

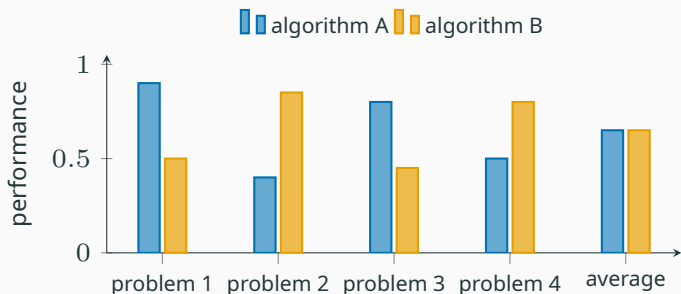
k -NN accuracy

$0.98 \rightarrow 0.555$
with noise features

- Notebook `03_pitfalls.ipynb`, section **D6**
- Practical consequences: feature selection, dimensionality reduction (Topic 7), more data

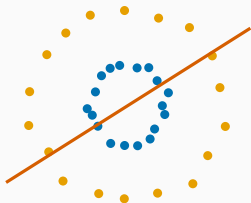
No Free Lunch

The No Free Lunch Theorem

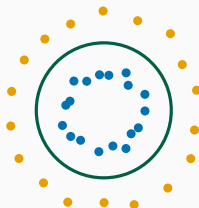


- **Theorem** (Wolpert, 1996): averaged over **all** possible problems, every learning algorithm has the same performance on unseen data
- Winning on some problems is always paid for by losing on others

Inductive Bias: the Assumptions a Model Makes



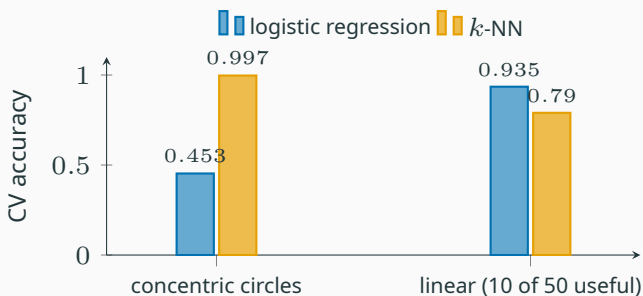
linear model: assumes a straight boundary **(fails)**



k -NN: assumes nearby points share labels **(works)**

- Every model carries **assumptions** about what patterns look like
- A model works when its assumptions **match** the problem

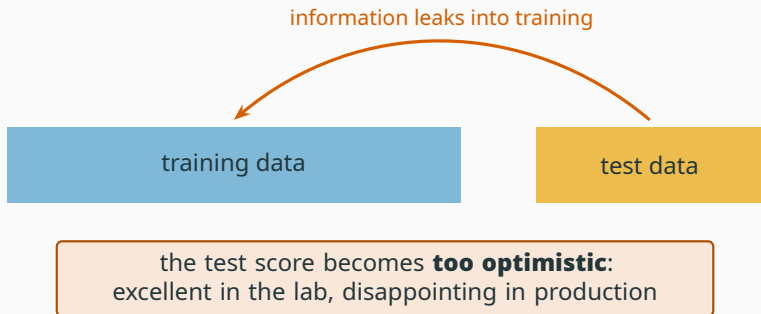
Live Demo D7: The Ranking Flips



- Notebook 03_pitfalls.ipynb, section **D7**
- No model is best everywhere: this is why the course covers **many model families**

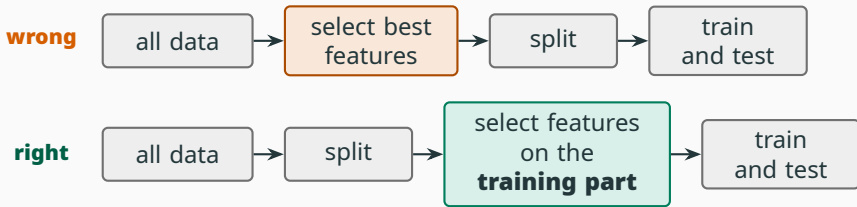
Data Leakage

What Is Data Leakage?



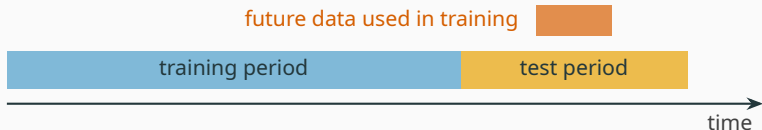
- **Leakage:** information that would not be available at prediction time reaches the model during training
- The test set stops being “unseen”, so it no longer measures generalization

Wrong and Right Pipelines



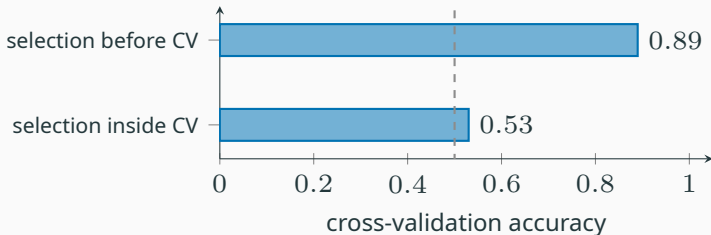
- In the wrong pipeline, the feature selection has **already seen** the test labels
- **Rule:** every step that learns from data (scaling, selection, imputation, tuning) goes **inside** the training part

Other Forms of Leakage



- **Temporal leakage:** random splits of time series let the model use the **future** to predict the past
- **Duplicates:** the same patient or image in both training and test
- **Target leakage:** a feature only known **after** the outcome (e.g. "treatment given" to predict the disease)

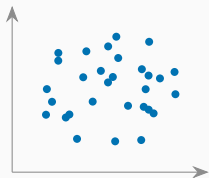
Live Demo D8: 89% Accuracy on Pure Noise



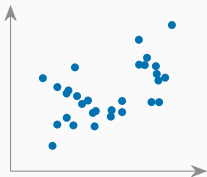
- 100 samples, 10 000 **random** features, **random** labels: the honest answer is 50%
- Selecting the 20 “best” features on all data first gives **89%** accuracy, from nothing
- Notebook 03_pitfalls.ipynb, section **D8**

Correlation Is Not Causation

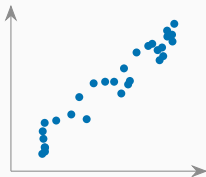
Correlation: Moving Together



$r \approx 0$
no relation



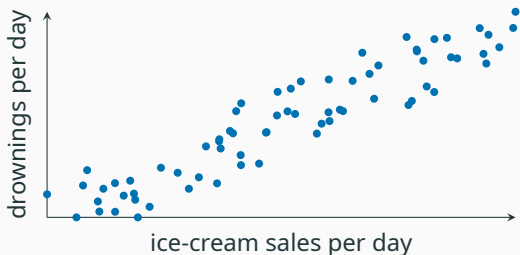
$r \approx 0.6$
moderate



$r \approx 0.95$
strong

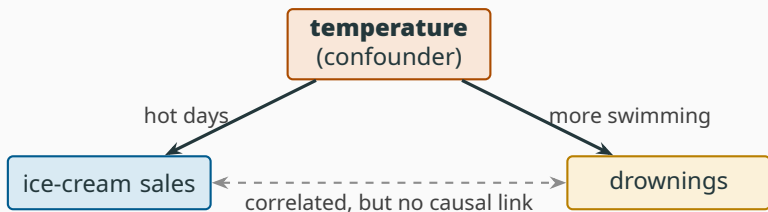
- **Correlation coefficient** r : a number from -1 to 1 describing how strongly two variables move **together**
- r near 0 : no linear relation; r near ± 1 : points lie close to a line
- Correlation says **nothing** about which variable causes which

Ice Cream and Drownings



- A beach town, one year of data (demo D9): correlation $r = 0.85$
- A model predicting drownings from ice-cream sales scores $R^2 = 0.73$: a **good predictor**
- Does ice cream **cause** drownings?

The Hidden Common Cause



- A **confounder** causes both variables and makes them move together
- Arrows show **causes**; the dashed line is only an **association**
- Adding temperature to the model: the weight of ice cream drops to about **0**

Prediction Versus Intervention

Prediction

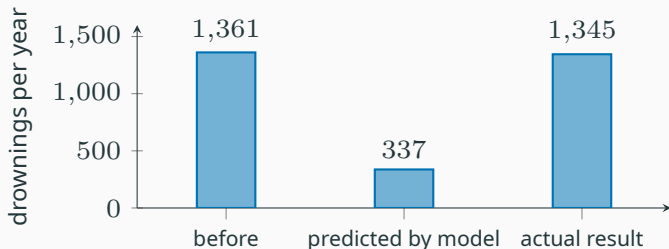
"I see high ice-cream sales;
how many drownings
should I expect today?"
correlation is enough

Intervention

"If I **force** sales to halve,
will drownings go down?"
needs the causal structure

- Models trained on observational data answer **prediction** questions
- Decisions that **change** the world are intervention questions
- Using a correlational model to make interventions leads to wrong decisions

Live Demo D9: Halving Ice-Cream Sales



- The model predicts that halving ice-cream sales prevents **75%** of drownings
- In the simulation nothing changes: temperature, the real cause, is the same
- Notebook `03_pitfalls.ipynb`, section **D9**

Shortcuts and Distribution Shift

Shortcut Learning



training: wolf in snow



test: wolf on grass

model says:
"husky"
it learned
"snow = wolf"

- Models take the **easiest** pattern that reduces training error: a **shortcut**
- The shortcut works on data like the training set and fails elsewhere

Documented Shortcuts

Pneumonia from X-rays

learned which
hospital
took the image

Skin cancer from photos

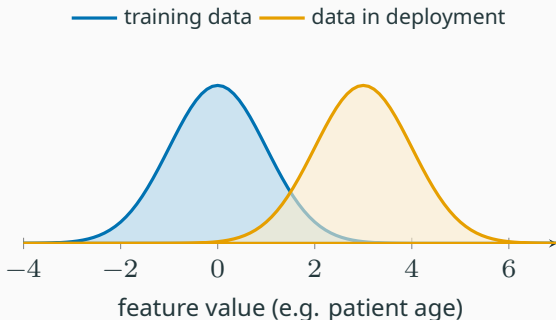
learned that a **ruler**
appears next to
suspicious lesions

Wolves vs. huskies

learned to
detect **snow**
in the background

- All of them looked excellent on a validation set from the **same** source
- See Geirhos et al., "Shortcut Learning in Deep Neural Networks", 2020

Distribution Shift



- Machine learning assumes training and deployment data come from the **same distribution**
- In practice the world **changes**: new users, new sensors, a new season
- Where the two distributions do not overlap, the model is **guessing**

Kinds of Shift

Covariate shift

$P(x)$ changes
a new camera
produces
darker images

Label shift

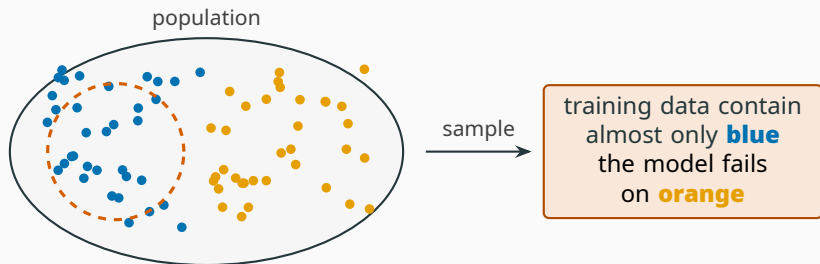
$P(y)$ changes
a disease becomes
more common
during an outbreak

Concept drift

$P(y | x)$ changes
fraudsters change
their tactics

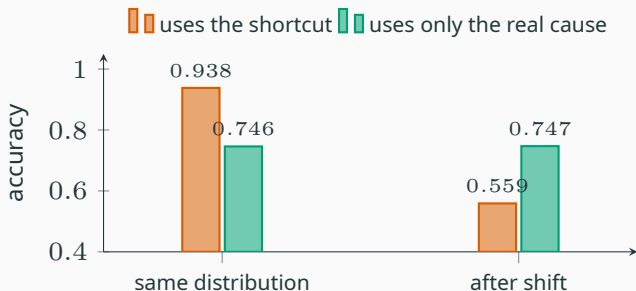
- $P(x)$: how often each input occurs; $P(y)$: how often each label occurs
- $P(y | x)$: the relationship between input and label, "the rule" itself

Sampling Bias



- **Sampling bias:** the training data do not represent the population where the model is used
- Under-represented groups receive **worse** predictions
- Historical data can also encode historical **discrimination**

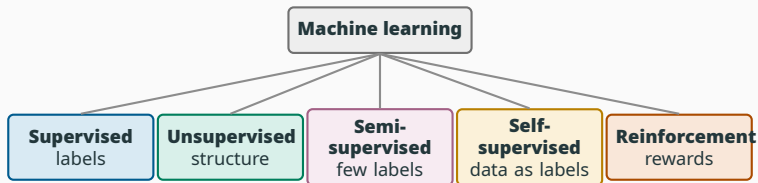
Live Demo D10: Shortcuts Under Shift



- The shortcut model looks **better** in validation, and collapses to near chance after the shift
- Only data from **deployment conditions** reveal the problem
- Notebook `03_pitfalls.ipynb`, section **D10**

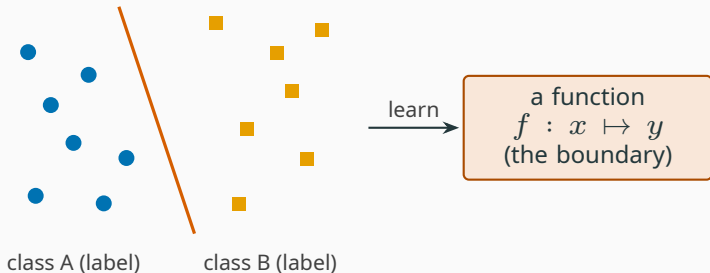
Learning Taxonomies

Where Does the Learning Signal Come From?



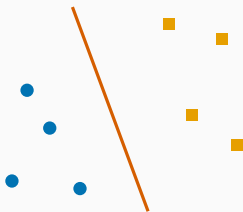
- Learning types differ in **what tells the model it is doing well**
- The same data can be used in several ways

Supervised Learning

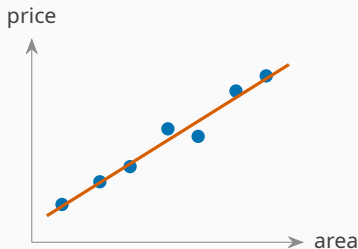


- Every training example comes with the **correct answer** y (the label)
- The model learns to map inputs x to outputs y
- The loss compares the prediction with the label

Classification and Regression



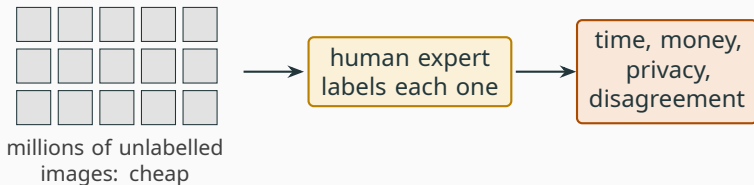
classification:
 y is a category
spam / not spam, digit 0–9



regression: y is a number
price, temperature

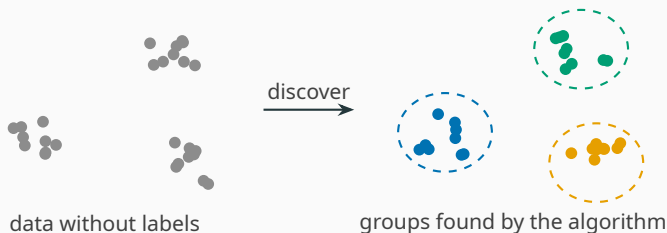
- Same idea, different kind of output
- Topics 3–6: linear and probabilistic models, SVM, neural networks, trees, k -NN, ensembles

The Price of Labels



- Labels often require **experts** (radiologists, lawyers, engineers)
- Experts **disagree**, and labels contain errors
- Most of the world's data has **no labels**: this motivates the other learning types

Unsupervised Learning

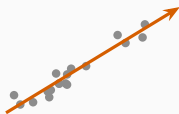


- Only inputs x , **no labels** y
- The model finds **structure**: groups, directions, hidden sources
- Harder to evaluate: there is no correct answer to compare with

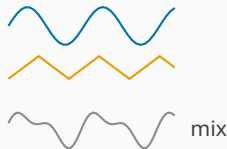
Unsupervised Tasks



clustering



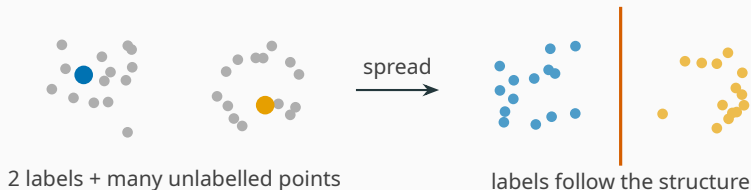
dimensionality reduction



blind signal separation

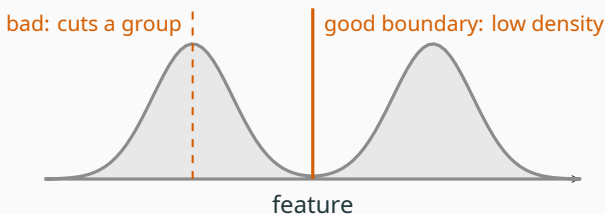
- **Clustering:** group similar examples (Topic 8)
- **Dimensionality reduction:** keep the few directions that matter (Topic 7)
- **Blind signal separation:** recover the sources hidden in a mixture (Topic 7)

Semi-Supervised Learning



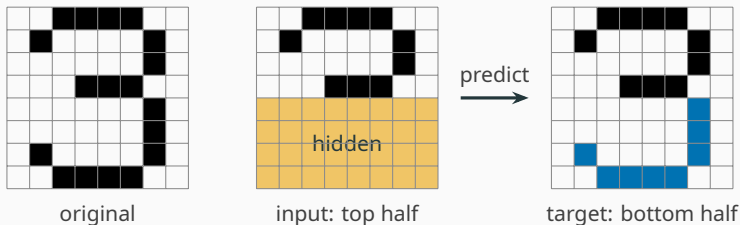
- A **few** labelled examples and **many** unlabelled ones
- Unlabelled data reveal the **shape** of the data; the few labels name the groups
- Methods: **self-training** (pseudo-labelling), label propagation on a similarity graph

The Cluster Assumption



- **Assumption:** points in the same dense group usually share a label
- So decision boundaries should pass through **empty** regions
- When this assumption fails, unlabelled data can **hurt** instead of help

Self-Supervised Learning



- No human labels: the **targets are created from the data itself**
- Hide part of the input and train the model to predict it (a **pretext task**)
- To succeed, the model must learn how the data are **structured**

Pretext Tasks in Practice

Masked words

"the cat ???
on the mat"
predict "sat"
(Transformers)

Reconstruction

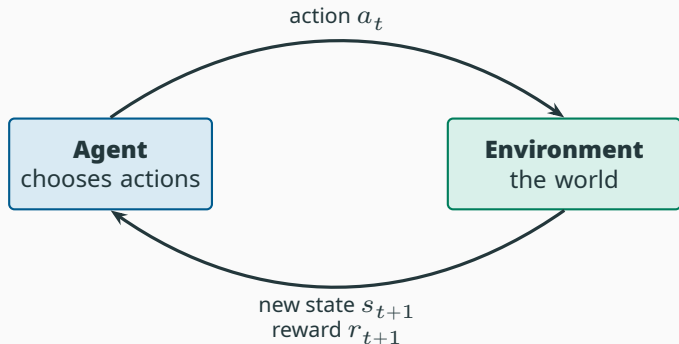
squeeze the input
through a bottleneck
and rebuild it
(AutoEncoders)

Predict in latent space

predict the
representation
of a hidden region
(JEPA)

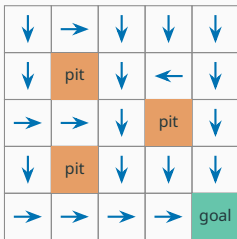
- The engine behind modern **foundation models**:
learning as compression, at scale
- Topic 9 covers these three families

Reinforcement Learning



- No dataset of correct answers: the agent **acts** and receives a **reward**
- s_t : the situation at time t ; a_t : the action taken; r_{t+1} : the reward received
- Goal: learn a **policy** (what to do in each state) that maximizes the total reward

Rewards, Exploration, and Exploitation



policy learned by Q-learning (demo D11)

Exploit: take the best action known so far

Explore: try a random action, it might be better

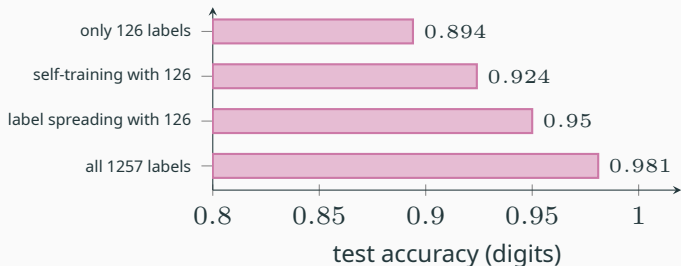
- Reward: $+1$ at the goal, -1 in a pit, -0.01 per step; nobody says which move is correct
- The agent must balance **exploration** and **exploitation** (Topic 10)

Summary of Learning Types

Type	Data	Learning signal	Topics
Supervised	(x, y) pairs	human labels	3–6
Unsupervised	only x	structure of the data	7–8
Semi-supervised	few (x, y) , many x	labels + structure	9
Self-supervised	only x	targets built from x	9
Reinforcement	interaction	rewards	10

- The **model families** (linear, trees, neural networks, ...) can be used in several of these settings
- The learning type describes **how** the model is trained, not **which** model it is

Live Demo D11: One Dataset, Many Signals



- Notebook `04_taxonomies.ipynb`: supervised, k -means clustering, **10% of the labels**, masked-half prediction
- **Reinforcement learning**: average return rises from -0.26 to $+0.64$ in 500 episodes

Wrap-Up

Key Takeaways

1. Compression

keep the pattern,
drop the noise

2. MDL

balance fit and
complexity

3. Optimization

finds the parameters

4. Evaluation

CV, intervals,
statistical tests

5. Pitfalls

bias-variance, curse,
leakage, causation

6. Learning types

where the signal
comes from

- These foundations apply to **every** model in the rest of the course

Next Class: Learning as Optimization

Blind optimization

random search, hill climbing,
genetic algorithms,
differential evolution, PSO

Gradient-based

derivatives and chain rule
by hand, gradient descent,
SGD, **JAX**

Lab: fit the same model with pyBlindOpt
and with gradient descent, then compare

- Learning as **search** over parameters: loss functions and search spaces

References

- C. E. Shannon, "A Mathematical Theory of Communication", *Bell System Technical Journal*, 1948
- A. N. Kolmogorov, "Three Approaches to the Quantitative Definition of Information", *Problems of Information Transmission*, 1965
- J. Rissanen, "Modeling by Shortest Data Description", *Automatica*, 1978
- D. H. Wolpert, "The Lack of A Priori Distinctions Between Learning Algorithms", *Neural Computation*, 1996
- T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997
- C. Nadeau and Y. Bengio, "Inference for the Generalization Error", *Machine Learning*, 2003
- D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, Cambridge University Press, 2003
- N. Silver, *The Signal and the Noise*, Penguin, 2012
- R. Geirhos et al., "Shortcut Learning in Deep Neural Networks", *Nature Machine Intelligence*, 2020